

NiceGUI 的 background_tasks 详细解析

NiceGUI 中的 `background_tasks` 是用于管理**异步后台任务**的核心机制，旨在让开发者能够在不阻塞 UI 主线程的前提下执行耗时操作（如网络请求、数据处理、定时任务等），保证前端界面的流畅性。以下从核心特性、使用方式、生命周期、注意事项等维度全面拆解：

一、核心定位与设计初衷

NiceGUI 基于 FastAPI + Vue 构建，UI 交互依赖主线程的事件循环（`asyncio`）。如果直接在 UI 回调（如按钮点击、输入框变更）中执行耗时同步操作，会导致界面卡顿、响应延迟。`background_tasks` 本质是对 `asyncio` 任务的封装，提供了简洁的 API 来创建、管理、终止后台任务，且天然适配 NiceGUI 的应用生命周期。

二、基础使用方式

1. 最简化使用（创建后台任务）

通过 `ui.run()` 启动应用时，可直接传入后台任务；或在运行时通过 `background_tasks.create()` 动态创建。

```
from nicegui import ui, background_tasks
import asyncio

# 定义耗时的后台任务函数（支持同步/异步函数）
async def long_running_task(seconds: int):
    for i in range(seconds):
        print(f"后台任务执行中: {i+1}/{seconds}")
        await asyncio.sleep(1)
    print("后台任务完成!")

# 方式1: 启动应用时传入初始后台任务
# ui.run(background_tasks=[long_running_task(5)])

# 方式2: 运行时动态创建（如按钮触发）
@ui.button("启动后台任务")
async def start_task():
    # create() 接收协程对象，自动加入事件循环
    task = background_tasks.create(long_running_task(5))
    # 可选: 记录任务ID, 用于后续管理
    ui.notify(f"后台任务已启动, ID: {task.id}")

ui.run()
```

2. 同步函数适配

如果后台逻辑是同步耗时函数（如 CPU 密集型计算），无需手动封装为异步，`background_tasks` 会自动将其放入线程池执行：

```
import time
from nicegui import ui, background_tasks

def sync_heavy_task(seconds: int):
    # 同步耗时操作（如数据处理、文件读写）
    time.sleep(seconds)
```

```
print("同步后台任务完成! ")

@ui.button("启动同步后台任务")
def start_sync_task():
    # 直接传入同步函数，自动异步执行
    background_tasks.create(sync_heavy_task(3))

ui.run()
```

三、任务管理核心 API

background_tasks 提供了一系列方法来管控任务生命周期：

方法	作用	示例
<code>create(coro_or_func, *args, **kwargs)</code>	创建后台任务 (支持协程 / 同步函数)	<code>task = background_tasks.create(long_running_task, 5)</code>
<code>get(task_id)</code>	通过 ID 获取任务对象	<code>task = background_tasks.get("task-123")</code>
<code>cancel(task_id)</code>	终止指定 ID 的任务	<code>background_tasks.cancel(task.id)</code>
<code>cancel_all()</code>	终止所有后台任务	<code>background_tasks.cancel_all()</code>
<code>list()</code>	获取所有活跃任务列表	<code>active_tasks = background_tasks.list()</code>

示例：任务终止与状态查询

```
from nicegui import ui, background_tasks
import asyncio

task_id = None

async def loop_task():
    try:
        while True:
            print("任务运行中...")
            await asyncio.sleep(1)
    except asyncio.CancelledError:
        print("任务被终止! ")

@ui.button("启动循环任务")
async def start_loop():
    global task_id
    task = background_tasks.create(loop_task())
    task_id = task.id
    ui.notify(f"任务启动, ID: {task_id}")
```

```

@ui.button("终止循环任务")
def stop_loop():
    if task_id:
        background_tasks.cancel(task_id)
        ui.notify("任务已终止")

@ui.button("查看活跃任务")
def show_tasks():
    tasks = background_tasks.list()
    ui.notify(f"当前活跃任务数: {len(tasks)}")
    for t in tasks:
        print(f"任务ID: {t.id}, 状态: {t.status}")

ui.run()

```

四、生命周期与注意事项

1. **应用退出时的任务处理**：默认情况下，NiceGUI 退出时会自动终止所有后台任务；若需优雅关闭（如完成收尾操作），可在任务中捕获 `asyncio.CancelledError`：

```

async def task_with_cleanup():
    try:
        # 核心逻辑
        await asyncio.sleep(10)
    except asyncio.CancelledError:
        # 收尾操作（如关闭文件、释放资源）
        print("任务被终止，执行清理...")
        raise # 必须重新抛出，确保任务正确终止

```

2. **线程安全**：`background_tasks` 的 API 是线程安全的，可在 UI 回调（主线程）、其他后台线程中调用。
3. **CPU 密集型任务建议**：对于纯 CPU 密集型操作（如大规模计算），建议结合 `asyncio.to_thread()` 或 `concurrent.futures.ThreadPoolExecutor`，避免阻塞事件循环：

```

from concurrent.futures import ThreadPoolExecutor
import asyncio

def cpu_heavy_calc():
    result = 0
    for i in range(10**8):
        result += i
    return result

async def run_calc():
    executor = ThreadPoolExecutor(max_workers=1)
    result = await asyncio.get_event_loop().run_in_executor(executor, cpu_heavy_calc)
    print(f"计算结果: {result}")

background_tasks.create(run_calc())

```

4. **任务状态说明**：任务对象包含 `status` 属性，可选值为：`pending`（待执行）、`running`（运行中）、

`cancelled`（已终止）、`finished`（已完成）。

五、典型应用场景

- 定时任务（如定时拉取数据、定时生成报表）；
- 异步 IO 操作（如网络请求、数据库查询）；
- 长时间运行的后台逻辑（如数据清洗、视频转码）；
- 实时数据推送（如 WebSocket 消息转发）。

background_tasks.create() 方法详细解析

`background_tasks.create()` 是 NiceGUI 中创建后台任务的**核心底层方法**，相比装饰器 `@background_tasks` 更灵活，支持动态控制任务创建时机、参数传递，是所有后台任务创建的基础入口。以下从方法定义、参数、返回值、使用场景、高级特性等维度全面拆解。

一、方法核心定义与定位

`create()` 是 `background_tasks` 模块的核心函数，作用是将**同步函数 / 异步协程**封装为后台任务，加入 `asyncio` 事件循环执行，且不阻塞调用线程（如 UI 主线程）。其底层逻辑是：

- 对异步协程（coroutine）：直接交由 `asyncio` 事件循环调度；
- 对同步函数（function）：自动封装为协程，通过 `asyncio.to_thread()` 放入线程池执行，避免阻塞事件循环。

二、方法参数详解

`create()` 的完整签名（简化版）：

```
def create(
    coro_or_func: Union[Coroutine, Callable],
    *args,
    **kwargs
) -> BackgroundTask:
    ...
```

参数	类型	说明	示例
<code>coro_or_func</code>	协程对象 / 可调用对象（函数、方法）	要执行的后台任务核心逻辑（必填）	协程： <code>long_running_task(5)</code> ；同步函数： <code>sync_task</code>
<code>*args</code>	任意类型	传递给 <code>coro_or_func</code> 的位置参数	<code>create(sync_task, 5, "test")</code>
<code>**kwargs</code>	任意类型	传递给 <code>coro_or_func</code> 的关键字参数	<code>create(async_task, seconds=5, msg="test")</code>

关键说明：

- 若传入的是**协程对象**（如 `async def` 函数调用后的结果），`create()` 直接调度执行；

- 若传入的是**同步函数 / 方法**（如普通 `def` 函数），`create()` 会自动将其封装为异步协程（通过 `asyncio.to_thread()`），无需手动转换；
- `args` 和 `kwargs` 会被直接传递给目标函数 / 协程，与普通函数调用的参数规则一致。

三、返回值：BackgroundTask 对象

`create()` 调用后返回一个 `BackgroundTask` 实例，该对象是任务的唯一管控入口，核心属性 / 方法如下：

属性 / 方法	类型	作用	示例
<code>id</code>	str	任务唯一标识（自动生成）	<code>print(task.id)</code> → <code>task-1698765432</code>
<code>status</code>	str	任务状态： <code>pending / running / cancelled / finished</code>	<code>if task.status == "running": ...</code>
<code>cancel()</code>	方法	终止当前任务（等效于 <code>background_tasks.cancel(task.id)</code> ）	<code>task.cancel()</code>
<code>result()</code>	方法	获取任务返回值（需任务完成，否则阻塞）	<code>res = await task.result()</code>
<code>exception()</code>	方法	获取任务执行中的异常（若有）	<code>err = task.exception()</code>

四、基础使用场景

1. 创建异步协程任务（最常用）

```
from nicegui import ui, background_tasks
import asyncio

# 定义异步任务函数
async def async_task(seconds: int, prefix: str):
    for i in range(seconds):
        print(f"{prefix} - 执行中: {i+1}/{seconds}")
        await asyncio.sleep(1)
    return f"{prefix} 任务完成"

# 动态创建后台任务（按钮触发）
@ui.button("启动异步任务")
async def start_async():
    # 传入协程对象（调用async_task返回的结果）+ 参数
    task = background_tasks.create(async_task(3, "异步任务1"))
    # 等待任务完成并获取返回值（可选）
    res = await task.result()
    ui.notify(res)

ui.run()
```

2. 创建同步函数任务

```
from nicegui import ui, background_tasks
import time

# 定义同步耗时函数
def sync_task(seconds: int):
    time.sleep(seconds)
    print("同步任务执行完成")
    return seconds

# 创建同步后台任务
@ui.button("启动同步任务")
def start_sync():
    # 传入同步函数 + 位置参数
    task = background_tasks.create(sync_task, 2)
    # 任务ID用于后续管理
    ui.notify(f"同步任务启动, ID: {task.id}")

ui.run()
```

3. 动态参数传递 (运行时确定参数)

`create()` 支持在调用时动态传递参数, 适合参数需根据 UI 输入、上下文变化的场景:

```
from nicegui import ui, background_tasks
import asyncio

async def dynamic_param_task(msg: str):
    await asyncio.sleep(1)
    print(f"动态参数任务: {msg}")

# 输入框获取参数, 动态创建任务
input_msg = ui.input("输入任务消息")

@ui.button("启动动态参数任务")
def start_dynamic():
    msg = input_msg.value or "默认消息"
    # 运行时传递输入框的参数
    background_tasks.create(dynamic_param_task(msg))

ui.run()
```

五、高级特性与细节

1. 任务取消的优雅处理

`create()` 创建的任务被取消时会抛出 `asyncio.CancelledError`, 需手动捕获以执行清理逻辑:

```
from nicegui import ui, background_tasks
import asyncio
```

```

task = None

async def cleanup_task():
    try:
        while True:
            print("任务运行中...")
            await asyncio.sleep(1)
    except asyncio.CancelledError:
        # 清理操作（如关闭文件、释放连接）
        print("任务取消，执行清理...")
        raise # 必须重新抛出，确保任务状态标记为cancelled

@ui.button("启动任务")
def start():
    global task
    task = background_tasks.create(cleanup_task())

@ui.button("取消任务")
def cancel():
    if task:
        task.cancel() # 等效于 background_tasks.cancel(task.id)

ui.run()

```

2. 批量创建任务与管理

`create()` 可批量调用创建多个任务，结合 `background_tasks.list()` 管理：

```

from nicegui import ui, background_tasks
import asyncio

async def batch_task(index: int):
    await asyncio.sleep(1)
    print(f"批量任务 {index} 完成")

@ui.button("创建10个批量任务")
def create_batch():
    task_ids = []
    for i in range(10):
        task = background_tasks.create(batch_task(i))
        task_ids.append(task.id)
    ui.notify(f"已创建 {len(task_ids)} 个任务")

@ui.button("终止所有批量任务")
def cancel_batch():
    background_tasks.cancel_all()
    ui.notify("所有任务已终止")

ui.run()

```

3. 与 UI 状态联动（非阻塞更新）

`create()` 创建的任务可安全更新 UI（NiceGUI 的 UI 操作线程安全），适合耗时任务中实时反馈进度：

```
from nicegui import ui, background_tasks
import asyncio

progress = ui.progress(0).props("indeterminate=False")

async def progress_task():
    for i in range(1, 101):
        progress.set_value(i) # 实时更新进度条
        await asyncio.sleep(0.05)
    progress.set_value(100)
    ui.notify("进度任务完成")

@ui.button("启动进度任务")
def start_progress():
    background_tasks.create(progress_task())

ui.run()
```

六、常见误区与注意事项

1. 传入函数 vs 协程对象：

- 错误：`background_tasks.create(async_task)`（仅传函数，未调用生成协程）；
- 正确：`background_tasks.create(async_task(5))`（调用函数生成协程对象）。

2. 同步函数的线程池限制：

`create()` 对同步函数默认使用 `asyncio` 的线程池（默认最大线程数由系统决定），若需自定义线程数，需手动指定 `executor`：

```
from concurrent.futures import ThreadPoolExecutor
import asyncio

def heavy_sync_task():
    ...

# 自定义线程池
executor = ThreadPoolExecutor(max_workers=4)
async def wrapper():
    await asyncio.get_event_loop().run_in_executor(executor, heavy_sync_task)

background_tasks.create(wrapper())
```

3. 任务结果的获取时机：

`task.result()` 是阻塞调用（异步），需在协程中等待，若任务未完成则会阻塞直到任务结束；若任务已取消 / 出错，调用 `result()` 会抛出对应异常。

4. 避免在任务中长时间持有 UI 锁：

虽然 UI 操作线程安全，但后台任务中频繁、长时间的 UI 更新仍可能导致界面卡顿，建议批量更新或降低更新

频率。

装饰器 `@background_tasks` 的详细使用解析

在 NiceGUI 中，`@background_tasks` 是针对**函数 / 方法**的装饰器形式，是 `background_tasks.create()` 的简化封装，核心作用是让被装饰的函数调用时自动以后台任务执行，无需手动调用 `create()`，进一步简化异步任务编写流程。

一、装饰器的核心特性

- 自动异步化**：被装饰的函数（无论同步 / 异步）调用时，会自动被封装为后台任务，加入 `asyncio` 事件循环，不阻塞调用线程（如 UI 主线程）；
- 调用方式无感知**：装饰后函数的调用方式与普通函数一致，无需额外传参或调整调用逻辑；
- 保留原函数功能**：装饰器仅修改执行方式，不改变函数的入参、返回值、逻辑等核心特性；
- 支持任务管理**：调用后返回的是后台任务对象，可通过该对象进行取消、查询状态等操作。

二、基础使用方式

1. 装饰异步函数

适用于本身就是协程的函数，装饰后调用时自动作为后台任务运行：

```
from nicegui import ui, background_tasks
import asyncio

# 用@background_tasks装饰异步函数
@background_tasks
async def async_background_task(seconds: int):
    for i in range(seconds):
        print(f"异步后台任务执行中: {i+1}/{seconds}")
        await asyncio.sleep(1)
    print("异步后台任务完成!")

# 调用方式与普通函数一致，自动以后台任务执行
@ui.button("启动异步后台任务")
def start_async_task():
    # 调用装饰后的函数，返回任务对象
    task = async_background_task(5)
    ui.notify(f"任务已启动, ID: {task.id}")

ui.run()
```

2. 装饰同步函数

同步耗时函数被装饰后，会自动被放入线程池异步执行，无需手动封装为协程：

```
from nicegui import ui, background_tasks
import time

# 用@background_tasks装饰同步函数
@background_tasks
```

```
def sync_background_task(seconds: int):
    time.sleep(seconds) # 同步耗时操作
    print("同步后台任务完成!")

@ui.button("启动同步后台任务")
def start_sync_task():
    # 调用后自动在后台执行，不阻塞UI
    task = sync_background_task(3)
    ui.notify(f"同步任务启动, ID: {task.id}")

ui.run()
```

三、装饰器与手动 create() 的对比

方式	写法	核心差异	适用场景
装饰器 @background_tasks	装饰函数后直接调用	简化调用流程，代码更简洁	函数需多次复用、调用逻辑简单
手动 create()	background_tasks.create(函数(参数))	灵活控制任务创建时机，支持动态传参	一次性任务、需动态调整任务参数

示例：两种方式对比

```
from nicegui import ui, background_tasks
import asyncio

# 方式1: 装饰器
@background_tasks
async def decorated_task():
    await asyncio.sleep(2)
    print("装饰器任务完成")

# 方式2: 手动create
async def manual_task():
    await asyncio.sleep(2)
    print("手动创建任务完成")

@ui.button("调用装饰器任务")
def call_decorated():
    decorated_task() # 直接调用，自动后台执行

@ui.button("手动创建任务")
def call_manual():
    background_tasks.create(manual_task()) # 需显式调用create

ui.run()
```

四、装饰器的进阶使用

1. 任务取消与状态管理

被装饰函数调用后返回的是任务对象，可通过该对象执行取消、查询状态等操作，与手动创建的任务完全一致：

```
from nicegui import ui, background_tasks
import asyncio

task = None

# 装饰循环任务
@background_tasks
async def loop_task():
    try:
        while True:
            print("循环任务运行中...")
            await asyncio.sleep(1)
    except asyncio.CancelledError:
        print("循环任务被终止")

@ui.button("启动循环任务")
def start_loop():
    global task
    task = loop_task() # 调用装饰函数，获取任务对象
    ui.notify("循环任务已启动")

@ui.button("终止循环任务")
def stop_loop():
    if task:
        background_tasks.cancel(task.id) # 通过ID取消任务
        ui.notify("循环任务已终止")

ui.run()
```

2. 装饰类方法

`@background_tasks` 也可用于类的方法，实现类内后台任务的封装：

```
from nicegui import ui, background_tasks
import asyncio

class TaskManager:
    @background_tasks # 装饰类方法
    async def run_task(self, name: str):
        for i in range(3):
            print(f"类方法任务 {name}: {i+1}/3")
            await asyncio.sleep(1)
        print(f"类方法任务 {name} 完成")

# 实例化并调用
tm = TaskManager()
```

```
@ui.button("启动类方法后台任务")
def start_class_task():
    tm.run_task("测试任务") # 调用后自动后台执行

ui.run()
```

3. 结合参数传递

装饰器支持函数接收任意参数，调用时直接传参即可，与普通函数无差异：

```
from nicegui import ui, background_tasks
import asyncio

@background_tasks
async def param_task(msg: str, interval: float):
    for i in range(2):
        print(f"{msg} - {i+1}")
        await asyncio.sleep(interval)

@ui.button("启动带参数的后台任务")
def start_param_task():
    # 传参方式与普通函数一致
    param_task("自定义消息", 0.5)

ui.run()
```

五、注意事项

1. **返回值处理**：后台任务运行在异步线程 / 协程中，被装饰函数的返回值无法直接同步获取（需通过回调、全局变量、事件等方式传递）：

```
from nicegui import ui, background_tasks
import asyncio

result = None

@background_tasks
async def task_with_return():
    global result
    await asyncio.sleep(2)
    result = "任务执行结果" # 通过全局变量传递返回值

@ui.button("启动任务并获取结果")
async def get_result():
    task_with_return()
    await asyncio.sleep(3) # 等待任务完成
    ui.notify(f"任务结果: {result}")

ui.run()
```

2. **异常处理**：被装饰函数内的异常不会阻塞主线程，但需手动捕获（否则仅打印到控制台，无 UI 提示）：

```

@background_tasks
async def task_with_error():
    try:
        1 / 0 # 模拟异常
    except Exception as e:
        ui.notify(f"任务出错: {e}", type="negative") # UI提示异常

@ui.button("启动带异常的任务")
def start_error_task():
    task_with_error()

```

3. **装饰器仅影响调用行为**：若仅定义被装饰函数但不调用，不会创建后台任务；多次调用会创建多个独立的后台任务。
4. **避免装饰高频调用函数**：若函数被高频调用（如每秒多次），需注意任务数量控制，避免创建过多活跃任务导致资源占用过高。

@background_tasks.await_on_shutdown 装饰器详细解析

`@background_tasks.await_on_shutdown` 是 NiceGUI 针对**后台任务优雅关闭**的专属装饰器，核心作用是标记需要在应用退出时“等待完成”的后台任务——默认情况下，NiceGUI 退出时会立即取消所有后台任务，而被该装饰器标记的任务，应用会等待其执行完毕后再退出，适用于需要收尾的关键任务（如数据落盘、连接关闭、事务提交等）。

一、核心设计背景

普通后台任务在应用退出（如按下 Ctrl+C、关闭进程）时，会被 `background_tasks` 强制取消（触发 `asyncio.CancelledError`），若任务包含“必须完成”的收尾逻辑（如将内存中的临时数据写入文件），强制取消会导致数据丢失 / 状态异常。`@background_tasks.await_on_shutdown` 正是为解决这一问题而生：

- 装饰后的任务，应用退出时**不会被取消**；
- 应用会阻塞退出流程，直到该任务执行完毕；
- 支持标记多个任务，应用会等待所有被标记的任务完成后再退出。

二、装饰器使用规则

1. 适用范围

仅能装饰**异步函数 / 协程**（`async def` 定义的函数），不支持同步函数（同步函数需手动封装为异步）。

2. 基础语法

```

from nicegui import background_tasks

@background_tasks.await_on_shutdown # 标记任务需等待退出
async def critical_task():
    # 关键逻辑：如数据落盘、连接关闭
    ...

```

3. 完整基础示例

```
from nicegui import ui, background_tasks
import asyncio
import time

# 被标记为“等待退出”的关键后台任务
@background_tasks.await_on_shutdown
async def cleanup_task():
    print("开始执行关键收尾任务...")
    # 模拟需要完成的收尾操作（如数据写入文件）
    await asyncio.sleep(3)
    print("关键收尾任务执行完毕！")

# 普通后台任务（退出时会被立即取消）
async def normal_task():
    try:
        while True:
            print("普通任务运行中...")
            await asyncio.sleep(1)
    except asyncio.CancelledError:
        print("普通任务被强制取消！")

# 启动任务
background_tasks.create(cleanup_task())
background_tasks.create(normal_task())

ui.run()
```

执行效果：

- 运行应用后按下 `Ctrl+C` 退出：
 - 普通任务立即被取消，打印“普通任务被强制取消！”；
 - 应用不会立即退出，而是等待 `cleanup_task` 执行完毕（3 秒后）；
 - `cleanup_task` 打印“关键收尾任务执行完毕！”后，应用才彻底退出。

三、进阶使用场景

1. 结合 `create()` 动态创建标记任务

装饰器可与 `background_tasks.create()` 结合，动态创建需要等待退出的任务：

```
from nicegui import ui, background_tasks
import asyncio

# 定义带参数的“等待退出”任务
@background_tasks.await_on_shutdown
async def save_data_task(file_path: str, data: str):
    print(f"开始将数据写入 {file_path}...")
    await asyncio.sleep(2) # 模拟文件写入耗时
    with open(file_path, "w") as f:
        f.write(data)
```

```

        print(f"数据已写入 {file_path}，任务完成")

# 动态创建任务（按钮触发）
@ui.button("启动数据保存任务")
def start_save_task():
    # 创建任务时自动继承“await_on_shutdown”标记
    background_tasks.create(save_data_task("data.txt", "关键业务数据"))

ui.run()

```

2. 多个标记任务的等待逻辑

应用会按“任务完成顺序”等待所有被标记的任务，直到全部完成后退出：

```

from nicegui import ui, background_tasks
import asyncio

@background_tasks.await_on_shutdown
async def task1():
    print("任务1启动（需等待退出）")
    await asyncio.sleep(2)
    print("任务1完成")

@background_tasks.await_on_shutdown
async def task2():
    print("任务2启动（需等待退出）")
    await asyncio.sleep(3)
    print("任务2完成")

# 启动两个标记任务
background_tasks.create(task1())
background_tasks.create(task2())

ui.run()

```

执行效果：

退出应用时，先等待 `task1` 完成（2 秒），再等待 `task2` 完成（再等 1 秒，总计 3 秒），最后应用退出。

四、关键注意事项

1. 避免无限循环任务：

被 `@background_tasks.await_on_shutdown` 标记的任务**不能是无限循环**（如 `while True`），否则应用会永远阻塞在退出流程中。若任务需循环执行，需增加退出触发条件：

```

import asyncio
from nicegui import background_tasks

shutdown_flag = False

@background_tasks.await_on_shutdown
async def loop_task_with_exit():

```

```

global shutdown_flag
while not shutdown_flag:
    print("循环任务运行中...")
    await asyncio.sleep(1)
print("循环任务正常退出")

# 模拟应用退出前触发标志
async def trigger_shutdown():
    global shutdown_flag
    await asyncio.sleep(5)
    shutdown_flag = True

background_tasks.create(loop_task_with_exit())
background_tasks.create(trigger_shutdown())

```

2. 同步函数的适配:

若需标记同步耗时任务为“等待退出”，需通过 `asyncio.to_thread()` 封装为异步函数：

```

from nicegui import background_tasks
import asyncio
import time

def sync_cleanup():
    # 同步收尾操作
    time.sleep(2)
    print("同步收尾任务完成")

# 封装为异步并标记
@background_tasks.await_on_shutdown
async def async_sync_cleanup():
    await asyncio.to_thread(sync_cleanup)

background_tasks.create(async_sync_cleanup())

```

3. 任务执行超时风险:

若被标记的任务执行时间过长（如网络请求超时、死循环），会导致应用无法正常退出，建议为任务添加超时控制：

```

from nicegui import background_tasks
import asyncio

@background_tasks.await_on_shutdown
async def task_with_timeout():
    try:
        # 超时控制：最多等待5秒，超时则终止
        await asyncio.wait_for(
            asyncio.sleep(10), # 模拟长时间任务
            timeout=5
        )
    except asyncio.TimeoutError:
        print("任务超时，强制终止")

```

```
print("任务收尾完成")
```

4. 与 `CancelledError` 兼容:

被标记的任务不会触发 `asyncio.CancelledError` (应用不会取消它), 因此无需捕获该异常; 若手动调用 `task.cancel()`, 仍会触发该异常 (装饰器仅影响应用退出时的行为)。

五、与普通任务的核心差异

特性	普通后台任务	<code>@await_on_shutdown</code> 标记任务
应用退出时行为	立即被取消 (触发 <code>CancelledError</code>)	不被取消, 应用等待其完成
退出阻塞	无, 应用立即退出	有, 阻塞直到任务完成
适用场景	非关键循环 / 临时任务	数据落盘、连接关闭、事务提交等关键收尾任务
支持函数类型	同步 / 异步	仅异步 (需封装同步函数)